

A System of Refusals

The SARSI Agents, Built

What Survived Implementation: One Loop, Twelve Planes,
and the Questions an Agent May Not Answer

Chengshuai Yang

NextGen PlatformAI C Corp.

`spiritai@platformai.org`

August 3, 2026

Abstract

Three prior papers designed a console: five agents behind one chat box, the same console once an account owns several machines, and the worker layer between a person's request and a session on their hardware. This paper reports the system that was actually built from them on a single host, and it differs from all three in a way worth stating first: **almost every module in it exists to refuse something.**

The loop is nine steps — a message is routed, a task is created, a plan is drafted, permissions are asked for, the plan is handed to a `sarsi-claude` session, the session works, an independent verifier rules, and a report returns. Two decisions inside it could reasonably have gone the other way and did not. A message does *not* start a session: it produces a task and a drafted plan and stops, because a message that silently spawned a governed session would make “I asked a question” and “I authorised work” the same gesture. And the manager routes by *creating a task on a worker* rather than by driving a session — a prohibition enforced at the point of execution, where the assignment call raises, rather than remembered in prose.

The paper's main contribution is the treatment of a question the design papers did not anticipate: **what happens when the session asks something.** A session stops for two reasons that look identical on a terminal and are opposite in kind. “*Which directory should I index?*” needs a fact the worker already holds. “*Do you want to proceed?*” asks whether the session may act. Conflating them is how an agent approves its own actions because it “knew what the owner would want.” The implemented order therefore checks for a permission gate *first* and declines outright — the language model is never shown a gate at all — then refuses any question naming an owner-reserved class however it is phrased, then answers only from the plan, the directive and its own notes, and otherwise says it does not know. Every outcome records the evidential rung its answer stands on, because an owner's answer and a worker's inference are different rungs and a record that cannot separate them cannot tell an auditor which answers were the owner's.

Owner precedence is likewise mechanical rather than conventional: entering interact mode *pauses the worker's steering*, because otherwise the worker types its next step into a terminal the owner is using and the two fight over the prompt.

A closing section lists what is not built, including one design question the implementation surfaced and did not answer: if a session may edit the plan it is judged by, the agent doing the work is editing its own standard.

Keywords: SARSI; multi-agent systems; agent governance; authority; implementation report; refusal as design; system design.

Contents

1	Introduction	3
1.1	Three designs, one implementation	3
1.2	The observation that organises the paper	3
2	The Loop	3
2.1	Nine steps	3
2.2	Two decisions that could have gone the other way	3
3	Twelve Planes, and What Each Refuses	4
4	The Question the Design Papers Did Not Anticipate	4
4.1	Two stops that look identical	4
4.2	The order is the mechanism	4
4.3	Every answer carries its rung	5
5	Owner Precedence Is Mechanical	5
6	The Console Above It	5
7	What Is Not Built	6
8	Conclusion	6

1 Introduction

1.1 Three designs, one implementation

This is an implementation report. Its predecessors specify a console fronted by a manager with five agents behind it [2], the same console once an account owns several machines [1], and the worker layer standing between a person’s request and a session on their machine [3]. What follows is the system built from them on one host, and the places where building it changed the design.

1.2 The observation that organises the paper

Most of this system is a set of things that will not happen

Twelve planes were implemented. Each is most economically described by what it *refuses*: a stranger’s message, a command to the manager, a plan without criteria, a control from a non-owning worker, a permission gate shown to a model, a secret leaving the machine, a verdict from the agent that did the work. The affirmative behaviour in each case is short and unsurprising. The refusals are where the design lives, and they are what took the effort to get right.

This is not a stylistic observation. A system specified by what it permits accumulates capability and discovers its boundaries later, usually from an incident. A system specified by what it refuses states the boundary first and lets capability grow inside it.

2 The Loop

2.1 Nine steps

$$U \rightarrow RT \rightarrow (MGR \mid AGT) \rightarrow TSK \rightarrow PLN \rightarrow [GRT] \rightarrow ASG \rightarrow CC \rightarrow VER \rightarrow REP$$

A message arrives (*U*) on either surface. The router (*RT*) decides which agent owns it and whether the sender may speak at all. Either the manager (*MGR*) or the addressed agent (*AGT*) takes it. A task is created (*TSK*), a plan is drafted (*PLN*), and the permissions that plan declares are asked for (*GRT*). The worker hands the plan — not the request — to a session (*ASG*); the session works (*CC*); an independent verifier rules (*VER*); a report returns (*REP*).

2.2 Two decisions that could have gone the other way

Rule 1 (A message does not start a session). *An inbound message produces a task and a drafted plan, and stops. The owner reads the plan and says start.*

The alternative — a message that spawns a governed session directly — is more responsive and collapses a distinction that must not collapse: it makes “*I asked a question*” and “*I authorised work*” the same gesture. The cost is one extra turn before anything runs. The benefit is that authorisation remains an act the owner performs rather than a side effect of conversing.

Rule 2 (The manager routes by creating a task, never by driving a session). *Creating a task on a worker is telling a worker to work, which the manager may do. Driving a session is executing, which it may not. The assignment call raises when the manager attempts it.*

The second half of that rule is the part that matters. A prohibition recorded only in documentation is a prohibition until someone adds a feature; a prohibition that raises at the point of execution is a property of the system.

3 Twelve Planes, and What Each Refuses

Table 1: The implemented planes, described by their refusals.

Plane	What it refuses
Registry	an agent absent from the registry file; a sender absent from the owner allowlist
Router	a stranger — dropped, counted, and never answered . Even a refusal would confirm that the bot exists and is listening
Manager	executing anything at all. It routes, and it answers
Worker	being commanded — a directive is evidence that something is wanted, and the worker decides whether it can be done here; and queueing silently, since an unplaceable directive that waits is indistinguishable from work in progress
Planner	a plan with no criteria. The verifier judges the plan’s <i>verified-when</i> lines, so a plan without them yields a verdict that means nothing
Task	a state that lies. <i>Awaiting-grant</i> , <i>waiting</i> and <i>awaiting-owner</i> each say what is true; none of them is <i>running</i>
Session	a control from any agent but the owning worker, and every control from the machine agent
Answer	a permission gate — the model is never shown one; a guess; and a reserved class however phrased (§4)
Vault	letting a secret leave the machine. Only a yes or a no crosses
Outward	transmitting anything but the bytes the owner approved
Verify	a verdict from the agent that produced the work
Modes	the worker typing while the owner holds the wheel (§5)

4 The Question the Design Papers Did Not Anticipate

4.1 Two stops that look identical

A `sarsi-claude` session halts for two reasons, and on a terminal they are the same event: text on a screen, waiting.

- “Which directory should I index?” — a request for a **fact**, and one the worker very likely already holds, written in the plan it drafted.
- “Do you want to proceed?” — a request for **authority**, asking whether the session may act.

The failure this section exists to prevent

Conflating the two is how an agent approves its own actions because it “knew what the owner would want.” The reasoning is locally plausible at every step, and the result is an agent spending authority nobody gave it.

4.2 The order is the mechanism

Rule 3 (The permission check runs first, and the model is never shown a gate). *Gate detection precedes every other step, and a detected gate is declined outright. The question is not passed to a language model for*

judgement, because a model shown a gate may reason its way to an answer. Gates remain on their existing path: the session’s ceiling, a standing grant the plan anticipated, or the owner.

Rule 4 (Phrasing does not launder a reserved class). *A question naming an owner-reserved class — money, consent, publishing, legal — goes to the owner regardless of its grammatical form. “Which card should I pay the invoice with?” is a money question wearing a choice’s clothes, and a gate detector tuned to “do you want to” will not catch it.*

Rule 5 (An answer is grounded or refused). *The worker answers only from the plan, the directive, and its own workspace notes, and otherwise returns an explicit I do not know and asks the owner. A guessed answer sends a session confidently in the wrong direction — and does so in the owner’s voice.*

4.3 Every answer carries its rung

Design principle 1 (An owner’s answer and a worker’s inference are different evidence). *An owner’s answer is authoritative for preferences and goals. A worker’s answer is an inference: provisional, and revisable. The record distinguishes them, because a record that cannot tell them apart cannot tell an auditor which answers the owner actually gave.*

This is the smallest change in the implementation and arguably the most important. The alternative — storing an answer as an answer — is indistinguishable in the happy case and unrecoverable in the case that matters, which is the one where something went wrong and the question is who decided.

5 Owner Precedence Is Mechanical

Three modes exist on a task. Their relationship is enforced rather than agreed.

Table 2: The three modes.

Mode	Behaviour
Guided	the owner steers <i>through the worker</i> , so their guidance lands under the same ownership check as the worker’s own. Nothing here reaches a session directly, which is what keeps the manager out of it
Interact	the owner takes the wheel. Steering pauses, the plan is marked stale, and the mode <i>reminds</i> the owner to attach to the session rather than relaying keystrokes into it
History	what was said, on either surface

Rule 6 (Interact pauses the worker’s steering). *Otherwise the worker continues typing its next step into a terminal the owner is using, and the two contend for the prompt. The pause is the mechanism by which the owner outranks the worker; without it, precedence is a convention that the scheduler does not implement.*

The plan is marked stale at the same moment, and for a separate reason: the owner is about to do something the plan does not describe. A plan surviving that would send the worker back through phases the owner has just overridden by hand.

6 The Console Above It

The console requires the agent runtime, installs it when absent, and opens gates the runtime keeps shut standalone so that agents may reach the manager and share a workspace. The dependency runs one way. A

missing prerequisite is therefore a *fault*, not a state, and code that catches it and continues with an empty result hides a broken installation behind an empty page.

The console’s surface over this system is deliberately read-only: three routes and one page, exposing agents, their tasks, and one task’s plan with its criteria and declared permissions. Two absences on that page are the design, and both are asserted by tests:

- **no progress bar**, because per-phase completion is recorded nowhere and a filled bar would be a number the page invented;
- **a plan that cannot be parsed says so**, because an empty phase list would read identically to “*no plan yet*” — a different and ordinary state.

7 What Is Not Built

1. **The worker and the session do not draft the plan together.** The planner drafts it and the assignment hands it down finished. The intended shape — the worker gives an initial plan, and the two refine it — surfaces a question the implementation did not answer and this paper will not pretend to: *if the session may rewrite the plan, the agent doing the work is editing the standard it is judged by*. The existing polish mechanism, in which the session proposes and the worker accepts, is the likely resolution.
2. **Owner precedence during that joint drafting is unspecified.** The steering pause covers interact mode only.
3. **No node carries a workspace.** This is the largest gap between the design papers and what was built, and it is worth naming precisely. The architecture specifies a bounded workspace per node — what that node may see, selected by salience — and the planner is the node that needs one most: it drafts from the request *alone*, with no access to what this agent has already tried, which plans were superseded, which permissions were refused before, or what the last verdict said. A planner without history redrafts the same plan after the same failure. The session-level design already carries the fix and the reason: a composer node there declared four history sources and was fed none of them, and could therefore re-issue an instruction that had just failed. The same defect exists here one level up, and the same remedy applies — feed the node, bound the selection, and count the overflow rather than truncating it silently.
4. **Settlement** — the usage fee, the exchange node, the initial non-exchangeable balance — is designed and unbuilt.
5. **The manager does not front the console.** It exists; it is not yet the way in.
6. **Everything here is one host.** The multi-machine form is designed [1] and not built.

8 Conclusion

The design papers in this series argue for properties. This one reports which properties survived being built, and the answer is: the ones that were implemented as refusals at the point of action, rather than as rules stated somewhere and honoured elsewhere.

The manager’s inability to drive a session is a property of the system because the call raises. Owner precedence is a property because interact mode pauses the worker. The ban on an agent answering its own gate is a property because the model is never shown the gate — not because the model is instructed to decline, which would make the property a matter of the model’s judgement on the day.

Everything in this system that is merely written down remains a hope. The work of building it was, in almost every case, the work of finding the place where a hope could be made into a refusal.

References

- [1] Chengshuai Yang. One account, many machines: Designing the multi-host SARSI console — per-host agents, host-indexed evidence, channels that can drop, and what a machine takes with it when it leaves. Preprint. The immediate predecessor of this paper, 2026.
- [2] Chengshuai Yang. One chat box, five agents: Designing the manager-fronted SARSI console — per-agent self-awareness, nested workspaces, and the four channels by which agents affect each other. Preprint, 2026.
- [3] Chengshuai Yang. One user, several machines: Designing the SARSI worker layer — server-side communicators without execution authority, five work agents, and a vault that only ever answers yes or no. Preprint, 2026.